

Ryan Diller
Professor Ali Simaei
MISY350
May 10, 2026

Individual Final Report

My project is the Patient Appointment Tracker. I was the only developer for this app. The purpose of the app is to help patients and doctors manage all their appointments in one place. Patients are able to register, log in, book appointments, view appointments, cancel or reschedule appointments, and use the AI assistant. Doctors are able to create available time slots, view their patient appointments, update appointment statuses, add notes, and view schedule statistics.

The Phase 1 version had the basic structure of the app application, including authentication, role-based dashboards, schedule management, appointment booking, and a hardcoded chat bot. The Phase 2/final version improved the design, added better scheduling tools, added OpenAI support, created object-oriented classes for data and chatbot handling, and added recurring appointment slot creation. In Phase 1 I contributed to the creation of the Patient Appointment Tracker app by creating the main structure and making sure the app had all the features it needed. Some of my Phase 1 contributions include: An authentication system which allowed users to create accounts, Role-based dashboards which organize the dashboards differently for patients and doctors suited for their needs, Appointment booking system that allowed patients to select a time slot doctors were available at, Appointment management which allowed patients to view, cancel and reschedule their appointments, and a basic chatbot that answers common questions. Some of the issues were that the app looked stale, the chatbot was limited, and there was no recurring schedule feature for the doctors, also the chat history was not saved for the user. In Phase 2 I worked on fixing these issues. I improved the UI and look of the app. I added in the OpenAI assistant. I added the recurring schedule creation which allowed doctors to create repeated appointment slots across multiple weeks, instead of manually. I also added in a weekly schedule calendar.

Changing the single time slot creation to recurring time slots required refactoring. The original code worked, but forced the doctors to manually book their times. Here is the original code:

```
def create_time_slot(doctor_email: str, date: str, time: str, duration_minutes: int = 30) -> dict:
    """Create an available time slot for a doctor"""
    schedules = load_schedules()
    slot_id = f"{doctor_email}_{date}_{time}".replace(" ", "_").replace(":", "-")

    new_slot = {
        "slot_id": slot_id,
        "doctor_email": doctor_email,
        "date": date,
        "time": time,
        "duration_minutes": duration_minutes,
        "status": "available",
        "created_at": datetime.now().isoformat()
    }

    schedules["schedules"].append(new_slot)
    save_schedules(schedules)

    return {"success": True, "message": "Time slot created", "slot_id": slot_id}
```

Refactoring the code allowed the doctors to be able to create repeated slots across a data range. This was a major usability improvement. Here is the final code:

```
def create_recurring_slots(doctor_email: str, start_date, end_date, weekdays: List[int],
                           time: str, duration_minutes: int, frequency_weeks: int = 1) -> dict:
    """Create recurring time slots over a date range."""
    if not weekdays:
        return {"success": False, "message": "Please select at least one weekday."}

    current_date = start_date
    created = 0
    while current_date <= end_date:
        week_number = ((current_date - start_date).days // 7)
        if week_number % frequency_weeks == 0 and current_date.weekday() in weekdays:
            result = create_time_slot(doctor_email, current_date.strftime("%Y-%m-%d"), time, duration_minutes)
            if result["success"]:
                created += 1
            current_date += timedelta(days=1)

    if created == 0:
        return {"success": False, "message": "No recurring slots were created. Check your date range or weekday selection."}

    return {"success": True, "message": f"Created {created} recurring slot{'s' if created != 1 else ''}."}
```

Also, changing the chat to store logs needed the code to be refactored. In Phase 1, the app answered one question at a time, but it did not save a real conversation. Here is the original code:

```
user_question = st.text_area("Your Question", placeholder="e.g., When is my next appointment?")

if st.button("Ask Assistant", use_container_width=True):
    if user_question:
        response = get_chatbot_response(user_question, patient_email, st.session_state.user["full_name"])
        st.info(response)
```

Changing the code made the assistant more complete because messages could now be saved and loaded. It also separated chat log storage into its own class, which is a better design. Here is the final code:

```
class ChatLoggerStore:
    def __init__(self, filepath: str):
        self.filepath = Path(filepath)

    def load_logs(self) -> list:
        if self.filepath.exists():
            with open(self.filepath, "r") as f:
                return json.load(f)
        return []

    def save_logs(self, logs: list) -> None:
        with open(self.filepath, "w") as f:
            json.dump(logs, f, indent=2)
```

An example of OOP throughout the app include the AppDataStore class. This is responsible for collecting appointment, schedule, and clinic information for the assistant. Here is the code:

```
class AppointmentDataStore:
    def __init__(self, patient_email: str):
        self.patient_email = patient_email

    def get_context(self) -> str:
        appointments = get_patient_appointments(self.patient_email)
        schedules = load_schedules()
        context = {
            "patient_email": self.patient_email,
            "appointments": appointments,
            "available_schedules": [
                s for s in schedules["schedules"] if s["status"] == "available"
            ],
            "clinic_info": {
                "name": "Patient Appointment Tracker Clinic",
                "hours": "Monday to Friday, 9:00 AM to 5:00 PM",
                "policies": [
                    "Bring a valid ID and insurance card",
                    "Arrive 10 minutes early",
                    "Contact staff if you need to reschedule"
                ]
            }
        }
        return json.dumps(context, indent=2)
```

The AppDataStore class improved the final version of the app by organizing patient and clinic data into one object. It collects and stores all the patient information so it can send it to the AI to answer questions. This shows encapsulation because the data collection code is kept inside one class. It also demonstrates separation of responsibilities because it prepares the data while the AI focuses on generating responses.

The design improvements from Phase 1 to Phase 2 made the app feel more professional and fully complete. While Phase 1 was functional, it looked plain. Phase 2 has custom styling that makes the app feel more complete and easier to read. Also, adding the weekly calendar view offers the users a more visual way to look at available appointments. All this made the app easier to use and understand for the users.

The AI assistant was one of the biggest improvements in the final version. In Phase 1, the assistant was useful but limited to a few preset keywords. In Phase 2, OpenAI support was added which made the assistant more flexible. I kept the hardcoded chatbot as a backup just in case the API fails.

Testing was very important because the app has two different user roles and many different actions for each. To test the patient side I went through the patient workflows, so I tested creating an account, logging in, booking an appointment, viewing appointments, canceling an appointment, rescheduling an appointment and asking the assistant questions. To test the doctor side I went through the doctor workflows, so I tested creating a single time slot, creating a recurring time slot, viewing the weekly schedule, deleting available time slots, viewing the patient roster, updating appointment status, adding appointment notes and checking schedule statistics.

This project helped me understand how a full app connects so much together and how it all fits in like a puzzle. The frontend design, the backend logic and data storage all come together. I learned that every app needs clear organization because the code can become hard to manage as more features are added. This is why it's important to have classes and separate the responsibilities. I also learned why refactoring is important. The final version showed better UI/UX design and the code was cleaner and more organized. Overall, working on this project has taught me a lot about development, and how to use Streamlit. It also made me understand JSON data storage, CRUD operations, and object-oriented programming.